Lab 5 Data Exchange with Local MQTT Broker

Drafted by Yuemin Ding

In many cases, remote MQTT brokers, e.g. the one of ThingSpeak, may not be available. A MQTT broker running in local network, which was developed in our previous practice task, would be helpful. Here, we use local MQTT brokers to support communications to interact with Arduino UNO R4 boards.

1. Arduino → Raspberry PI

Here, we may reuse the Arduino code developed before to send random sensor values to ThingSpeak but simply modify the MQTT broker address, username, password, cliendID (must be unique ID that is different from all the others).

Example Arduino code:

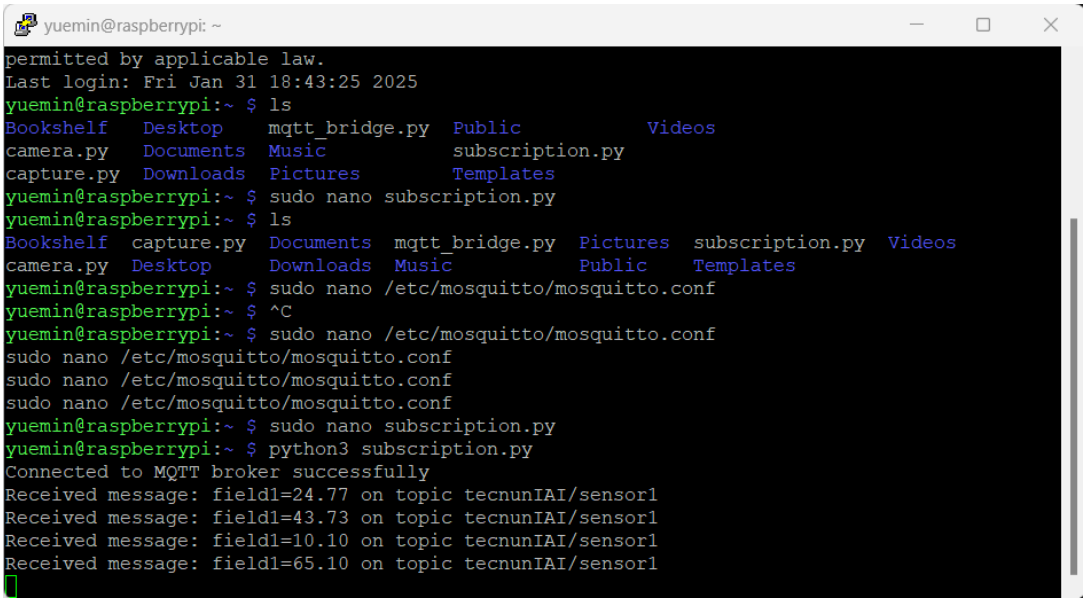
```
#include <WiFiS3.h>
#include <PubSubClient.h>

// Wi-Fi ssid and password
const char* ssid = "";
const char* password = "";

// MQTT broker
//char *server = "mqtt3.thingspeak.com";
IPAddress server(192, 168, 238, 20);
char *clientID = "";
char *topic = "";
char *userName = "";
char *mqttPassword = "";

// Declare WiFi and MQTT client
WiFiClient UNOR4Client;
PubSubClient client(server, 1883, NULL, UNOR4Client);

void startWifi() {
  Serial.print("\nUNOR4 connect with ");
  Serial.println(ssid);
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("Starting wifi connect...");
    WiFi.begin(ssid, password);
    delay(2000);
  }
  Serial.println("Wifi connected!");
```

```
  Serial.print("UNOR4 IP address: ");
  Serial.println(WiFi.localIP());
}

void connectBroker() {
  // User and password configured in the broker
  if (client.connect(clientID, userName, mqttPassword)) { //clientID,
username, password
    Serial.println("ThingSpeak connected!");
  }
}

void setup()
{
  Serial.begin(115200);
  startWifi();
  connectBroker();
}

unsigned long previousMillis = 0;
const long interval = 20000;  // 20 seconds
void loop()
{
  if (!client.connected()) {
    Serial.println("MQTT broker disconnected, reconnecting ...");
    connectBroker();  // If not connected, attempt to reconnect
    delay(5000);
  }
  else{
    client.loop();
    unsigned long currentMillis = millis();
    // Run only if interval has passed
    if (currentMillis - previousMillis >= interval) {
      previousMillis = currentMillis;
      float sensorValue1 = random(0,10000)/100.0;
      float sensorValue2 = random(0,10000)/100.0;
      char msg[50];
      sprintf(msg, "field1=%.2f&field2=%.2f", sensorValue1,
sensorValue2);
      client.publish(topic, msg);
      Serial.print("Published to MQTT broker:");
      Serial.println(msg);
    }
  }
```

```
}
```

The Python code for MQTT subscription can be reused as well. Make sure that the MQTT topic to which Arduino UNO R4 is publishing has been included in the Python file.

Here is what you may get from the Raspberry PI side.



2. Raspberry PI → Arduino

So far, we have tested how to publish messages from Arduino UNO to the MQTT broker but have not done it the other way around. Theoretically, if Arduino UNO subscribes to an MQTT topic, it will also receive messages from the broker.

Refer to the following code for Arduino UNO, it subscribes to an MQTT topic to receive messages. Depending on which message is received, it prints different messages to the SerialMonitor. For instance, if the message “green” is received, it prints "Turning on GREEN LED" to the Serial Monitor. In this way, we can control the output messages by publishing messages on the same topic.

Example Arduino code to receive mqtt messages:

```
#include <WiFiS3.h>
#include <PubSubClient.h>

// Wi-Fi ssid and password
const char* ssid = " ";
const char* password = " ";

// Your Raspberry Pi IP, in this format
//char *server = "mqtt3.thingspeak.com";
IPAddress server(192, 168, 238, 20);
char *clientID = "sensor2";
```

```
char *topic = "tecnunIAI/control1";
char *userName = "tecnunIAI";
char *mqttPassword = "tecnunIAI";

// Callback function, listen to the incoming message
void callback(char* topic, byte* payload, unsigned int length) {

  Serial.print("Message received on topic: ");
  Serial.println(topic);

  // Convert byte array to String
  String message;
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  Serial.print("Message: ");
  Serial.println(message);

  if (message.equals("green")) {
    Serial.println("Turning on GREEN LED");
  } else if (message.equals("red")) {
    Serial.println("Turning on RED LED");
  } else if (message.equals("blue")) {
    Serial.println("Turning on BLUE LED");
  } else if(message.equals("off")) {
    Serial.println("Turning off LEDs");
  } else {
    Serial.println("Unknown command.");
  }
}

WiFiClient UNOR4Client;
PubSubClient client(server, 1883, callback, UNOR4Client);

void startWifi() {
  Serial.print("\nUNOR4 connect with ");
  Serial.println(ssid);
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("Starting wifi connect...");
    WiFi.begin(ssid, password);
    delay(2000);
  }
```

```
  Serial.println("Wifi connected!");
  Serial.print("UNOR4 IP address: ");
  Serial.println(WiFi.localIP());
}

void connectBroker() {
  // User and password configured in the broker
  if (client.connect(clientID, userName, mqttPassword)) {      //clientID,
username, password
    client.subscribe(topic); //UNOR4 subscribe topic. See callback
function
    Serial.println("Broker connected!");
  }
}

void setup()
{
  Serial.begin(115200);
  startWifi();
  connectBroker();
}

void loop()
{
  if (!client.connected()) {
    Serial.println("MQTT broker disconnected, reconnecting ...");
    connectBroker();  // If not connected, attempt to reconnect
    delay(5000);
  }
  else{
    client.loop();
  }
}
```

Here are the outputs you may have.

Send mqtt messages from Raspberry PI:

```
yuemin@raspberrypi: ~
Received message: field1=65.86 on topic tecnunIAI/sensor1
Received message: field1=61.04 on topic tecnunIAI/sensor1
^CTraceback (most recent call last):
  File "/home/yuemin/subscription.py", line 27, in <module>
    client.loop_forever()
  File "/usr/lib/python3/dist-packages/paho/mqtt/client.py", line 1756, in loop_forever
    rc = self._loop(timeout)
         ^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3/dist-packages/paho/mqtt/client.py", line 1150, in _loop
    socklist = select.select(rlist, wlist, [], timeout)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt

yuemin@raspberrypi:~ $ mosquitto_pub -h localhost -u tecnunIAI -P tecnunIAI -t "tecnunIAI/co
ntrol1" -m "green"
yuemin@raspberrypi:~ $ mosquitto_pub -h localhost -u tecnunIAI -P tecnunIAI -t "tecnunIAI/co
ntrol1" -m "red"
yuemin@raspberrypi:~ $ mosquitto_pub -h localhost -u tecnunIAI -P tecnunIAI -t "tecnunIAI/co
ntrol1" -m "off"
yuemin@raspberrypi:~ $ mosquitto_pub -h localhost -u tecnunIAI -P tecnunIAI -t "tecnunIAI/con
trol1" -m "blue"
yuemin@raspberrypi:~ $
```

Output from Arduino:

```
COM20
Send

UNOR4 connect with VodafonePM0103
Starting wifi connect...
Wifi connected!
UNOR4 IP address: 192.168.238.184
Broker connected!
Message received on topic: tecnunIAI/control1
Message: green
Turning on GREEN LED
Message received on topic: tecnunIAI/control1
Message: red
Turning on RED LED
Message received on topic: tecnunIAI/control1
Message: off
Turning off LEDs
Message received on topic: tecnunIAI/control1
Message: blue
Turning on BLUE LED

Autoscroll  Show timestamp  Newline  115200 baud  Clear output
```

3. Arduino->Arduino through Rasp MQTT

Remember that Arduino UNO also supports publishing messages to an MQTT topic. If we use another Arduino UNO board, e.g. Arduino board 2, to send messages to the same MQTT topic, the messages printed by Arduino board 1 will change accordingly.

Referring to the following code for Arduino board 2, which randomly picks a command from {“green”, “blue”, “red”, “off”} and publishes it to Raspberry PI MQTT broker. The broker further forwards the command to Arduino board 1, which outputs different messages according to the command received. In this way, we can link two Arduino boards through data exchange.

Example Arduino code to send mqtt commands:

```
#include <WiFiS3.h>
#include <PubSubClient.h>

// Wi-Fi ssid and password
const char* ssid = " ";
const char* password = " ";

// MQTT broker
//char *server = "mqtt3.thingspeak.com";
IPAddress server(192, 168, 238, 20);
char *clientID = "sensor1";
char *topic = "tecnunIAI/control1";
char *userName = "tecnunIAI";
char *mqttPassword = "tecnunIAI";

// Declare WiFi and MQTT client
WiFiClient UNOR4Client;
PubSubClient client(server, 1883, NULL, UNOR4Client);

void startWifi() {
  Serial.print("\nUNOR4 connect with ");
  Serial.println(ssid);
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("Starting wifi connect...");
    WiFi.begin(ssid, password);
    delay(2000);
  }
  Serial.println("Wifi connected!");
  Serial.print("UNOR4 IP address: ");
  Serial.println(WiFi.localIP());
}

void connectBroker() {
  // User and password configured in the broker
  if (client.connect(clientID, userName, mqttPassword)) { //clientID,
username, password
    Serial.println("ThingSpeak connected!");
  }
}

void setup()
{
```

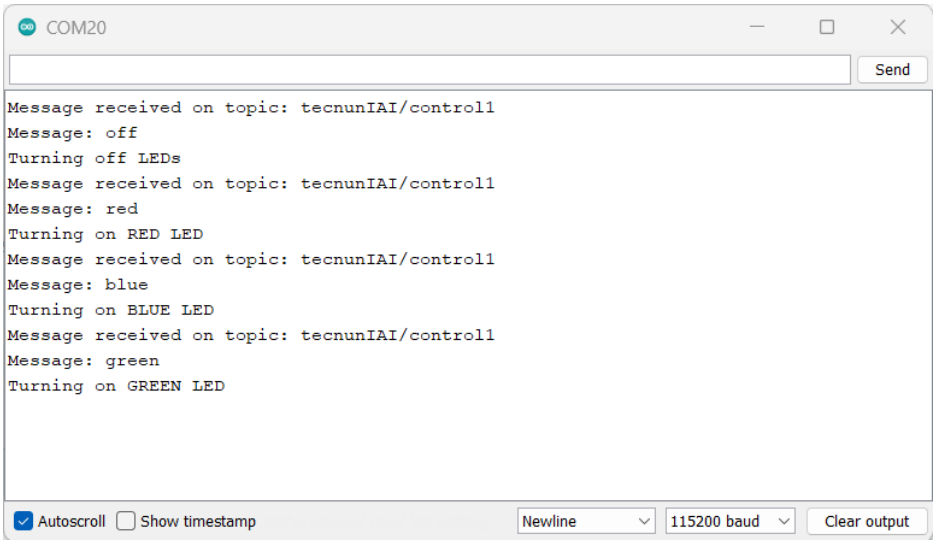
```
  Serial.begin(115200);
  startWifi();
  connectBroker();
}

unsigned long previousMillis = 0;
const long interval = 20000;  // 20 seconds
void loop()
{
  if (!client.connected()) {
    Serial.println("MQTT broker disconnected, reconnecting ...");
    connectBroker();  // If not connected, attempt to reconnect
    delay(5000);
  }
  else{
    client.loop();
    unsigned long currentMillis = millis();
    // Run only if interval has passed
    if (currentMillis - previousMillis >= interval) {
      previousMillis = currentMillis;

      char *commands[] = {"green","blue","red", "off"};
      int numCommands = sizeof(commands) / sizeof(commands[0]);  // Get the
number of strings
      int randomIndex = random(0, numCommands);  // Pick a random index

      char msg[50];
      sprintf(msg, commands[randomIndex]);
      client.publish(topic, msg);
      Serial.print("Published to MQTT broker:");
      Serial.println(msg);
    }
  }
}
```

Here are the outputs you may have:

For Arduino board 2 (the one sending random commands)

```
Published to MQTT broker:green

UNOR4 connect with VodafonePM0103
Starting wifi connect...
Wifi connected!
UNOR4 IP address: 192.168.238.127
ThingSpeak connected!
Published to MQTT broker:off
Published to MQTT broker:red
Published to MQTT broker:blue
Published to MQTT broker:green
```

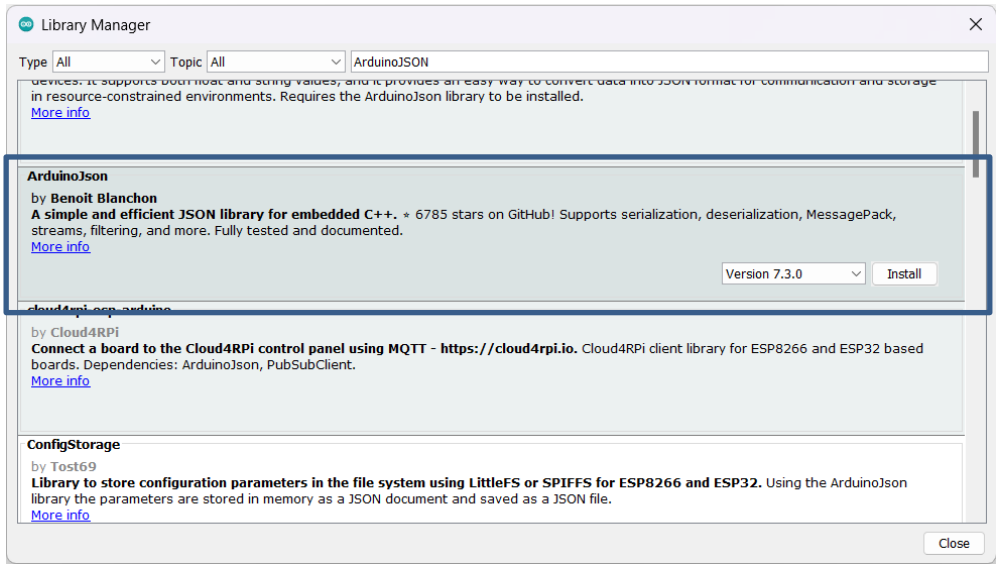
For Arduino board 1 (the one being controlled to print different messages)



4. Arduino JSON format and Data Processing on Raspberry

So far, we have been using Arduino boards to send messages to Raspberry PI and ThingSpeak with MQTT. The data are directly formulated as strings. Another commonly used approach is to pack the sensing data as JSON, which is a lightweight, standardized, and unified data format for data exchange between devices and applications.

To pack the sensing data using JSON, there is a library in Arduino to help us to do so. Search for 'ArduinoJson' in the library manager and install it.



Example Arduino code to pack the sensor data as below and send through MQTT.

```
{
  "temperature": {
  "value": 25,
  "unit": "Celsius"
  },
  "humidity": {
  "value": 60,
  "unit": "%"
  }
}
```

```
#include <WiFiS3.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>


// Wi-Fi ssid and password
const char* ssid = "";
const char* password = "";

// MQTT broker
//char *server = "mqtt3.thingspeak.com";
IPAddress server(192, 168, 238, 20);
char *clientID = "";
char *topic = "";
char *userName = "";
char *mqttPassword = "";

// Declare WiFi and MQTT client
WiFiClient UNOR4Client;
PubSubClient client(server, 1883, NULL, UNOR4Client);

void startWifi() {
  Serial.print("\nUNOR4 connect with ");
  Serial.println(ssid);
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("Starting wifi connect...");
    WiFi.begin(ssid, password);
    delay(2000);
  }
  Serial.println("Wifi connected!");
  Serial.print("UNOR4 IP address: ");
```

```
  Serial.println(WiFi.localIP());
}

void connectBroker() {
  // User and password configured in the broker
  if (client.connect(clientID, userName, mqttPassword)) { //clientID,
username, password
    Serial.println("MQTT broker connected!");
  }
}

void setup()
{
  Serial.begin(115200);
  startWifi();
  connectBroker();
}

unsigned long previousMillis = 0;
const long interval = 20000;  // 20 seconds
void loop()
{
  if (!client.connected()) {
    Serial.println("Broker disconnected, reconnecting ...");
    connectBroker();  // If not connected, attempt to reconnect
    delay(5000);
  }
  else{
    client.loop();
    unsigned long currentMillis = millis();
    // Run only if interval has passed
    if (currentMillis - previousMillis >= interval) {
      previousMillis = currentMillis;
      float temp = random(0,5000)/100.0;
      float humd = random(0,10000)/100.0;

      /*Prepare a json file like below
       {
         "sensor":"jsonSensor1",
         "temp": {
         "value": 25,
         "unit": "Celsius"
         },
         "humd": {
```

```
        "value": 60,
        "unit": "%"
        }
      }
     */

      StaticJsonDocument<200> jsonDoc;
      jsonDoc["sensor"] = "jsonSensor1";
      jsonDoc["temp"]["value"] = temp ;
      jsonDoc["temp"]["unit"] = "Celsius";
      jsonDoc["humd"]["value"] = humd;
      jsonDoc["humd"]["unit"] = "%";

      char msg[256];
      //sprintf(msg, "field1=%.2f&field2=%.2f", sensorValue1,
sensorValue2);

      char buffer[256];

      // After serialization:
{"sensor":"temperature","value":25,"unit":"Celsius"}
      serializeJson(jsonDoc, msg);  //convert json doc to C string

      client.publish(topic, msg);
      Serial.print("Published to MQTT broker:");
      Serial.println(msg);
    }
  }
}
```

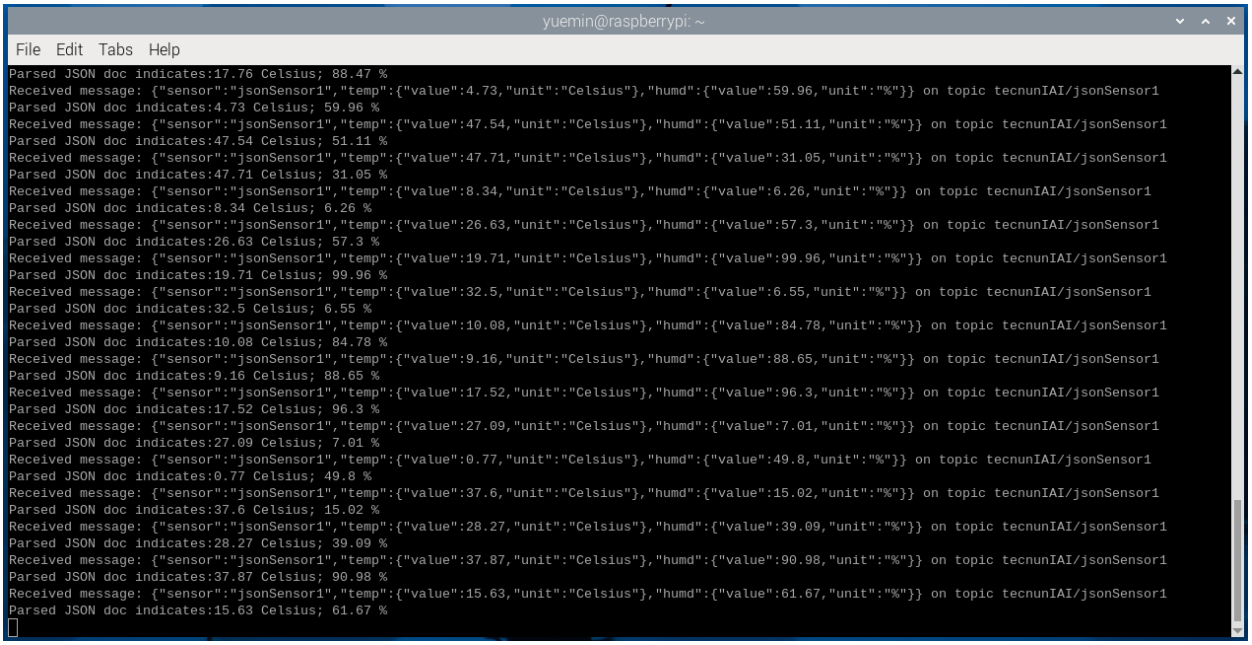
If we run the code, we can see the JSON file published to the MQTT broker through the Arduino Serial Monitor. It looks quite similar to a Python dictionary.

After receiving the JSON file, we can simply process it and extract the data with Python. For that, the json library would be very helpful.

Example Python code to parse the JSON file from Arduino:

```
import paho.mqtt.client as mqtt
import json

def on_message(client, userdata, message):
    print(f"Received message: {message.payload.decode()} on topic
{message.topic}")
    fields = json.loads(message.payload.decode("utf-8"))
    tempValue = fields['temp']['value']
    tempUnit = fields['temp']['unit']
    humdValue = fields['humd']['value']
    humdUnit = fields['humd']['unit']
    print(f"Parsed JSON doc indicates:{tempValue} {tempUnit}; {humdValue}
{humdUnit}")

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connected to MQTT broker successfully")
        client.subscribe("tecnunIAI/jsonSensor1")
    else:
        print(f"Failed to connect, return code {rc}")

BROKER = "localhost"
PORT = 1883
USERNAME = ""
PASSWORD = ""

client = mqtt.Client()
client.username_pw_set(USERNAME, PASSWORD)
client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, 60)

client.loop_forever()
```

If everything goes well, you will be able to see the JSON file and the data extracted below.

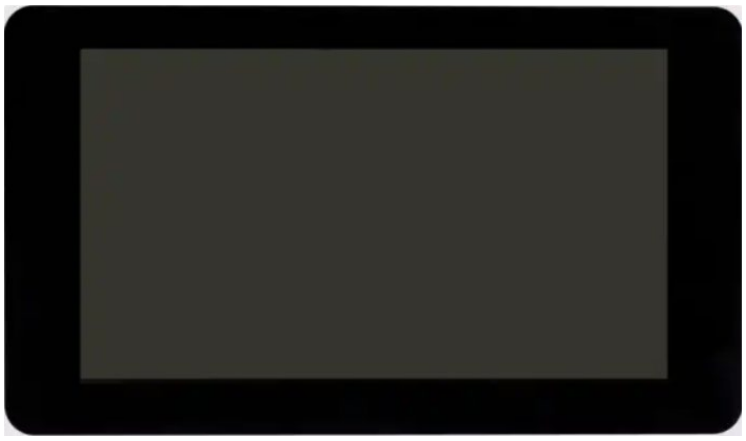


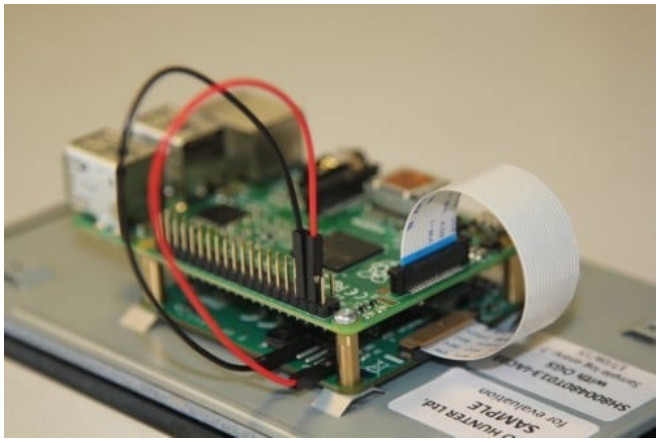
Note: it is also possible to parse a JSON file directly at Arduino with the same library. It would be helpful to send values to control the smart devices, e.g. remotely set the room temperature to a thermostat. If interested, you may figure out how to do so by yourself.

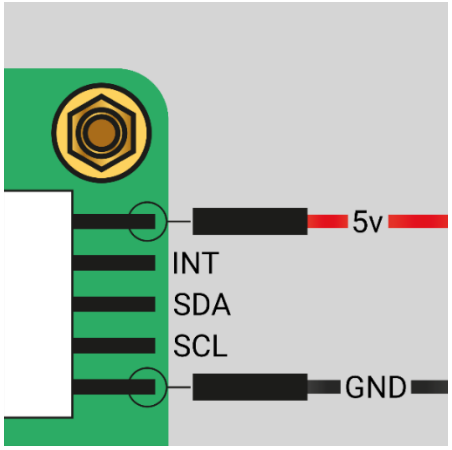
5. Changing PI to a Local HMI

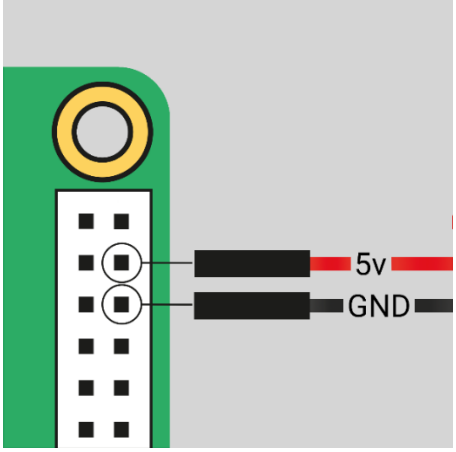
Processing the data with a local MQTT broker could be very helpful, especially when a local user interface is required for monitoring, control, data analysis, and decision-making. For instance, a local HMI is needed in a smart house application to collect data from all the sensors and/or control smart appliances. We may simply add a touchscreen to Raspberry PI to convert it to a local HMI device. To do so, refer here: https://www.raspberrypi.com/documentation/accessories/display.html.

(Here, we will not cover how to develop a user interface. Later in this course, Grafana will be taught to introduced for user interface development)









Note: Raspberry PI is super sensitive. Before connecting the touch screen, remember to switch it off and double check the power and GND pins are connected correctly.

Practice task:

Change the previous practice task, in which we used a Potentiometer to control the color of RGB LED. Here are the requirements:

- Use Arduino board 1 to monitor the changes of the potentiometer, format the data using JSON, and publish the JSON file to the MQTT broker on Raspberry PI
- Parse the JSON file to obtain the values of the potentiometer and print the value to the screen. Based on the value, send the control command to Arduino board 2.
- Use Arduino board 2 to control the color of the RGE LED
- When the output of the Potentiometer changes, the color of the RGB LED changes accordingly (0-40% → Green; 40%-80%V→Yellow, and 80%-100%V → Red).
- Document the process and submit it with the code.